

Comparative Evaluation of Scheduling Algorithms and the Proposed AJIE Algorithm

Aisha Tomaz, Icaro Cavalcante, Elilúcio Teixeira, Samuel Jackson

Software Engineering

Center for Science and Technology

Federal University of Cariri

Juazeiro do Norte, Ceará, Brazil

Abstract—Process scheduling is a fundamental mechanism of multiprogrammed operating systems, influencing metrics such as turnaround time, throughput, and fairness. This paper presents the implementation and experimental evaluation of a discrete-event simulator written in C, comparing the First-Come, First-Served (FCFS), Round Robin (RR), and Non-Preemptive Priority policies against the proposed AJIE algorithm, which is based on deterministic aging. The algorithms are evaluated across four workload scenarios, using 1,000 processes and 100 independent seeds per scenario, taking context-switch overhead into account. Performance is analyzed through turnaround time, number of context switches, slowdown, and Jain’s fairness index, with 95% confidence intervals. The results show that, in the unbalanced-priority scenario, AJIE raises Jain’s index from 54.65% to 90.72% relative to Non-Preemptive Priority, while performing significantly fewer context switches than Round Robin. These results indicate that AJIE offers a favorable trade-off between fairness and scheduling overhead in non-preemptive environments.

Index Terms—Process Scheduling, Discrete-Event Simulation, Aging, Jain’s Fairness Index, Context Switching.

I. INTRODUCTION AND MOTIVATION

The Central Processing Unit (CPU) process management and scheduling subsystem is one of the core components of modern operating system architecture [1], [2]. In multiprogrammed environments, the scheduler must arbitrate the allocation of the processor among multiple concurrent processes that compete for computational cycles and access to Input/Output (I/O) devices. The effectiveness of this mechanism directly impacts critical system metrics, such as mean turnaround time, waiting time in ready queues, throughput, and fairness in resource distribution [3], [4].

However, the design of scheduling algorithms faces fundamental trade-offs:

- Simple arrival-order policies, such as First-Come, First-Served (FCFS), operate with minimal context-switch overhead but suffer severely from the convoy effect when long processes hold the CPU, harming short and I/O-bound processes.
- Time-sharing policies, such as Round Robin (RR) [5], guarantee responsiveness and bound maximum waiting time, but impose cumulative context-switch penalties (C_{sw}) that degrade overall performance when the time quantum is poorly calibrated.
- Static priority policies allow business urgency to be differentiated across tasks, but introduce the starvation

pathology, in which low-priority processes may be indefinitely deferred if the inflow of high-priority tasks is continuous [1].

Although classical algorithms such as Highest Response Ratio Next (HRRN) and Shortest Job First (SJF) seek to minimize waiting time and mitigate starvation, such approaches fundamentally depend on prior knowledge of the duration of the next CPU burst — an oracle assumption that is unrealistic in general-purpose systems [2]. Moreover, in critical operational niches — such as High-Performance Computing (HPC) clusters, atomic batch financial-processing routines, and soft real-time embedded systems without timer-based preemption support — interrupting a task mid-execution is prohibitive or incurs excessive checkpointing I/O overhead [6].

In this context, this work investigates the empirical and analytical behavior of different scheduling policies within a discrete-time simulation environment built in C. The central objective is to compare classical strategies (FCFS, RR, and Non-Preemptive Priority) with an original policy proposed by the team, named **AJIE** (*Aisha-Jackson-Icaro-Elilucio*). The study evaluates the proposed algorithm’s ability to deterministically eradicate starvation in a non-preemptive regime without temporal foresight, quantifying its fairness gains through Jain’s index applied to the slowdown metric [7] and analyzing the impacts of context-switch cost ($C_{sw} > 0$).

II. DESCRIPTION OF THE PROPOSED SYSTEM MODEL

To ensure methodological rigor and full reproducibility, the simulator implements a discrete-time model based on a simulation clock (*simulation ticks*), where all state transitions, I/O requests, and hardware overheads operate deterministically, controlled by a pseudo-random generator parameterized by a seed [8].

A. Process Model and Attributes

Each simulated process P_i is instantiated as an independent entity carrying the minimal and sufficient set of metadata for scheduler decision-making:

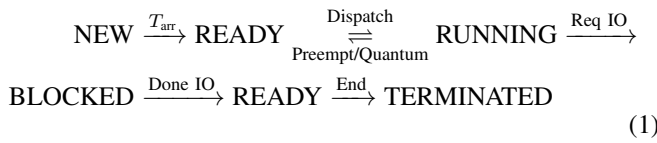
- **Unique Identifier (*‘pid’*)**: Sequential integer $i \geq 0$.
- **Arrival Instant ($T_{arrival}$)**: Tick at which the process is admitted into the system and inserted into the ready queue. Arrivals are modeled via a stochastic process with exponentially distributed inter-arrival intervals ($\Delta t \sim \text{Exp}(\lambda)$),

avoiding the artificial collapse of simultaneous queuing at instant $T = 0$.

- **Base Static Priority (P_{base}):** Priority level assigned at creation, following the standardized convention from 1 (lowest priority) to $L = 10$ (highest priority).
- **Burst Sequence:** Linked list alternating between CPU bursts and I/O operations: $(\text{CPU}_1, \text{IO}_1, \text{CPU}_2, \dots, \text{CPU}_k)$. The end of the last CPU burst marks the completion of the process.
- **Accumulated Metrics:** Counters for context switches suffered ($N_{\text{sw},i}$), total time spent in the waiting queue, and completion instant ($T_{\text{completion},i}$).

B. Life Cycle and State Transitions

The life cycle of each process follows the strict state machine illustrated conceptually below:



Only processes in the READY state are eligible to receive the processor. Processes blocked on peripheral operations remain outside the ready queue until the exact duration stipulated for the device has elapsed.

C. I/O Modeling and Device Concurrency

Input/Output modeling adopts perfect concurrency among devices (multiple I/O operations in parallel), managed through a minimum priority queue (*Min-Heap*) indexed by completion tick: $T_{\text{unblock}} = T_{\text{current}} + \text{io_duration}$. When the simulation clock reaches T_{unblock} , the process is immediately transferred from the BLOCKED state to the tail of the READY queue, becoming eligible for the next scheduling decision. The same I/O infrastructure is used indistinctly for all evaluated algorithms.

D. Context-Switch Cost Modeling

The context switch is modeled as an explicit global parameter $C_{\text{sw}} > 0$ measured in ticks. The cost is incurred whenever the CPU disassociates from the current process and dispatches a new one:

- 1) **Quantum Expiration (Round Robin):** RUNNING $\rightarrow$ READY.
- 2) **I/O Blocking:** RUNNING $\rightarrow$ BLOCKED.
- 3) **Execution Termination:** RUNNING $\rightarrow$ TERMINATED.

During the interval $[t, t + C_{\text{sw}})$, the CPU remains in the CONTEXT_SWITCHING state, 100% occupied with administrative overhead and unavailable for process progress. Consecutive transitions in which the CPU remains idle (IDLE $\rightarrow$ IDLE) do not incur any time cost.

III. DESCRIPTION OF THE IMPLEMENTED ALGORITHMS

Three reference algorithms and the team's proposed algorithm were implemented:

A. First-Come, First-Served (FCFS)

A pure non-preemptive policy that services requests in strict order of arrival at the ready queue (T_{queue}). The running process retains the CPU until its CPU burst ends or an I/O request occurs. It is immune to starvation, but vulnerable to severe degradation under long CPU-bound processes.

B. Round Robin (RR)

A preemptive time-slicing policy. Each process admitted into the circular ready queue executes for at most one time quantum q . If the CPU burst is not completed within q , the process is preempted, re-inserted at the end of the queue, and the CPU incurs the context-switch cost C_{sw} .

C. Non-Preemptive Priority Scheduling

A non-preemptive algorithm that selects for execution the ready process with the highest numerical priority value ($P_i \in [1, L]$). Processes with identical priorities are tie-broken by FCFS order of entry into the ready queue. Once dispatched, the process runs uninterrupted until it yields the CPU.

D. Proposed Algorithm: AJIE

The AJIE algorithm was designed to solve starvation in non-preemptive environments without resorting to time oracles. The scheduler maintains L FCFS queues ($Q_1, \dots, Q_L$), where the index corresponds to the process's current effective priority (P_{eff}).

1) **Selection Rule and Aging:** At each dispatch decision (Algorithm 1), the scheduler extracts the process at the head of the highest-priority non-empty queue. Immediately after the selection, all remaining processes waiting in queues $Q_1, \dots, Q_{L-1}$ that were already present at the start of the round have their effective priority incremented by +1, being promoted to Q_{j+1} . Processes that arrive during the execution of the current process enter their base queue $Q_{P_{\text{base}}}$ and only become eligible for aging in the subsequent round.

2) **Anti-Starvation Guarantee and Complexity: Theorem (Deferral Ceiling):** Given a system with L levels, any process P_i with base priority P_{base} will be deferred at most $B_{\text{max}} = L - P_{\text{base}}$ times before reaching the maximum level Q_L , where it is serviced in strict FIFO order. The complexity per scheduling decision is $O(N_{\text{ready}})$, operating in $O(1)$ when implemented via linked-queue pointers.

Algorithm 1 AJIE Scheduling

Require: Set of priority queues $[Q_1, \dots, Q_L]$, parameters L and C_{sw}

Ensure: Process P_{sel} selected for non-preemptive execution

```

1:  $k \leftarrow L$ 
2: while  $k \geq 1$  and  $Q_k.isEmpty()$  do
3:    $k \leftarrow k - 1$ 
4: end while
5: if  $k < 1$  then
6:   return NULL {CPU idle}
7: end if
8:  $P_{sel} \leftarrow Q_k.dequeue()$  {FIFO dispatch at the highest level}
9: for  $j = 1$  to  $L - 1$  do
10:   $N_j \leftarrow Q_j.size()$ 
11:  for  $i = 1$  to  $N_j$  do
12:     $P \leftarrow Q_j.dequeue()$ 
13:    if  $P.admittedBeforeCurrentRound$  then
14:       $P.P_{eff} \leftarrow \min(P.P_{eff} + 1, L)$ 
15:       $Q_{j+1}.enqueue(P)$  {Aging promotion}
16:    else
17:       $P.admittedBeforeCurrentRound \leftarrow \text{true}$ 
18:       $Q_j.enqueue(P)$ 
19:    end if
20:  end for
21: end for
22: return  $P_{sel}$ 

```

IV. EXPERIMENTAL METHODOLOGY

A. Workload Generation and Reproducibility

Workload generation uses the *xoshiro256*** pseudo-random generator, seeded via the *SplitMix64* algorithm from a 64-bit integer seed [8]. For each experimental run (S, C) , where S is the seed and C is the scenario, the exact sequence of generated processes — including arrival instants, priorities, and CPU/I/O bursts — is **100% identical and invariant** across all four evaluated algorithms.

B. Standardized Simulation Scenarios

The evaluation covers four mandatory scenarios that stress distinct system characteristics:

- 1) **Scenario 1: Balanced Random (balanced_random):** A heterogeneous workload simulating general-purpose systems, with a balanced distribution of short/long bursts and I/O requests.
- 2) **Scenario 2: I/O-Bound (io_bound):** Highly interactive processes with short CPU bursts ($[1, 8]$ ticks) and frequent, long I/O requests ($[20, 60]$ ticks).
- 3) **Scenario 3: CPU-Bound (cpu_bound):** Heavy computational workloads with long CPU bursts ($[40, 200]$ ticks) and rare I/O requests ($[0, 2]$).
- 4) **Scenario 4: Unbalanced Priorities (priority_unbalanced):** A highly asymmetric distribution with 85% high-priority processes ($[8, 10]$) and 15% low-priority processes ($[1, 3]$), designed to induce starvation in static schedulers.

TABLE I: Parameter Matrix of the Experimental Scenarios

Parameter	S1: Balanced	S2: I/O-Bound	S3: CPU-Bound	S4: Unbalanced
CPU Burst (ticks)	[5, 50]	[1, 8]	[40, 200]	[5, 50]
Number of I/O Requests	[1, 5]	[6, 15]	[0, 2]	[1, 5]
I/O Duration (ticks)	[10, 30]	[20, 60]	[5, 15]	[10, 30]
Inter-arrival (Δt)	~ 5	~ 3	~ 12	~ 4
Priorities (P_i)	[1, 10] unif.	[1, 10] unif.	[1, 10] unif.	85% $\in [8, 10]$, 15% $\in [1, 3]$

C. Scale and Statistical Rigor

Each scenario was run over a workload of $N = 1,000$ processes per run across $M = 100$ independent seeds, totaling 1,600 complete simulation runs. Aggregated metrics are reported as the sample mean accompanied by the 95% Confidence Interval (95% CI), computed via the Student's t -distribution:

$$CI_{95\%} = \bar{X} \pm t_{M-1, 0.975} \cdot \frac{s}{\sqrt{M}} \quad (2)$$

where $\bar{X}$ is the sample mean, s is the standard deviation, and $M = 100$.

D. Formal Evaluation Metrics

The algorithms are quantified according to three mandatory primary metrics:

- 1) **Mean Turnaround Time ($\bar{T}_{\text{turnaround}}$):** Time elapsed between the admission instant and process completion:

$$T_{\text{turnaround}, i} = T_{\text{completion}, i} - T_{\text{arrival}, i} \quad (3)$$

- 2) **Total Context Switches (N_{sw}):** Absolute number of process replacements on the CPU during the total run.
- 3) **Jain's Fairness Index on Slowdown (J_{slowdown}):** The individual slowdown S_i measures the proportional penalty relative to the ideal minimum time $T_{\text{ideal}, i} = \sum \text{CPU}_i + \sum \text{IO}_i$:

$$S_i = \frac{T_{\text{turnaround}, i}}{T_{\text{ideal}, i}} \quad (\text{with } S_i \geq 1.0) \quad (4)$$

Jain's index [7] assesses the degree of fairness among processes:

$$J(S) = \frac{\left(\sum_{i=1}^N S_i\right)^2}{N \cdot \sum_{i=1}^N S_i^2} \times 100\% \quad (5)$$

Values close to 100% indicate homogeneous treatment of all processes, while low values reveal disproportionate discrimination.

V. EXPERIMENTAL RESULTS

Tables II, III, IV, and V consolidate the mean values and confidence intervals obtained across the 1,600 simulations performed. Figures 1, 2, 3, and 4 illustrate the visual behavior with 95% CI error bars.

TABLE II: Mean turnaround (ticks) (mean $\pm$ 95% CI across seeds)

Scenario	AJIE	FCFS	Priority	RR
balanced_random	82113.28 $\pm$ 269.06	82318.78 $\pm$ 268.62	56770.92 $\pm$ 181.01	124061.00 $\pm$ 398.57
cpu_bound	141326.83 $\pm$ 649.02	142007.82 $\pm$ 647.04	116147.66 $\pm$ 429.42	252854.23 $\pm$ 1055.26
io_bound	63253.79 $\pm$ 117.87	60461.61 $\pm$ 117.89	36222.09 $\pm$ 62.68	70090.06 $\pm$ 141.12
priority_unbalanced	82662.01 $\pm$ 320.04	82872.17 $\pm$ 322.40	57213.38 $\pm$ 177.80	124414.86 $\pm$ 468.74

TABLE III: Context switches (count) (mean $\pm$ 95% CI across seeds)

Scenario	AJIE	FCFS	Priority	RR
balanced_random	3004.3 $\pm$ 7.8	4004.3 $\pm$ 7.8	4004.3 $\pm$ 7.8	29093.5 $\pm$ 68.8
cpu_bound	998.9 $\pm$ 5.0	1998.9 $\pm$ 5.0	1998.9 $\pm$ 5.0	60747.6 $\pm$ 189.1
io_bound	10503.0 $\pm$ 15.7	11503.0 $\pm$ 15.7	11503.0 $\pm$ 15.7	17264.5 $\pm$ 25.2
priority_unbalanced	2999.9 $\pm$ 9.3	3999.9 $\pm$ 9.3	3999.9 $\pm$ 9.3	29050.4 $\pm$ 78.3

TABLE IV: Mean slowdown (mean $\pm$ 95% CI across seeds)

Scenario	AJIE	FCFS	Priority	RR
balanced_random	522.788 $\pm$ 0.891	523.590 $\pm$ 0.856	426.054 $\pm$ 1.871	786.410 $\pm$ 1.008
cpu_bound	615.082 $\pm$ 1.560	617.254 $\pm$ 1.580	663.348 $\pm$ 3.101	1112.478 $\pm$ 1.088
io_bound	138.117 $\pm$ 0.144	132.828 $\pm$ 0.143	83.939 $\pm$ 0.185	154.254 $\pm$ 0.185
priority_unbalanced	527.919 $\pm$ 0.882	528.749 $\pm$ 0.905	431.308 $\pm$ 1.498	790.656 $\pm$ 1.019

TABLE V: Jain's fairness index on slowdown (%) (mean $\pm$ 95% CI across seeds)

Scenario	AJIE	FCFS	Priority	RR
balanced_random	90.55 $\pm$ 0.15	90.60 $\pm$ 0.15	54.45 $\pm$ 0.33	95.73 $\pm$ 0.03
cpu_bound	77.71 $\pm$ 0.23	77.68 $\pm$ 0.22	46.97 $\pm$ 0.27	96.34 $\pm$ 0.03
io_bound	98.02 $\pm$ 0.02	97.68 $\pm$ 0.02	67.28 $\pm$ 0.11	97.27 $\pm$ 0.03
priority_unbalanced	90.72 $\pm$ 0.16	90.79 $\pm$ 0.16	54.65 $\pm$ 0.30	95.74 $\pm$ 0.04

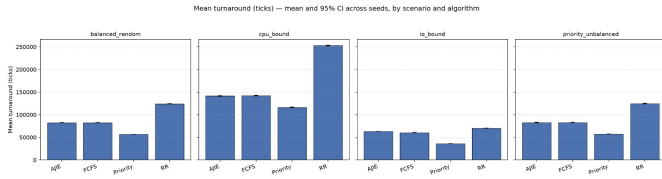


Fig. 1: Comparison of mean turnaround among algorithms across the 4 scenarios (with 95% CI error bars).

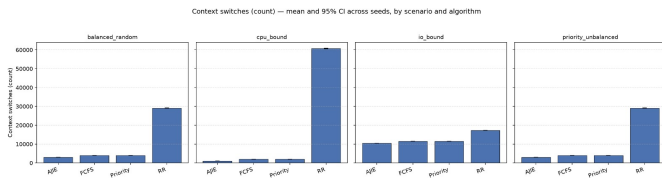


Fig. 2: Total accumulated context switches per scheduling policy.

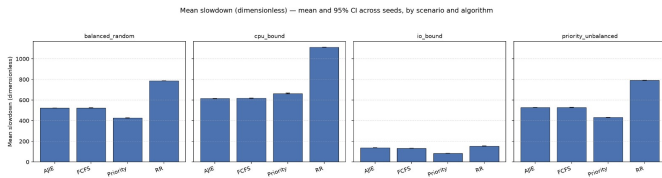


Fig. 3: Jain's fairness index applied to process slowdown (0% to 100%).

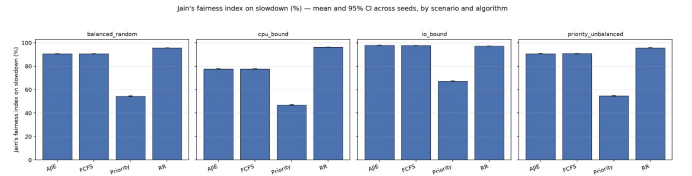


Fig. 4: Jain's fairness index applied to process slowdown (0% to 100%).

VI. DISCUSSION OF RESULTS

The comparative analysis allows the theoretical hypotheses of operating systems to be confronted with the empirical behavior observed in the simulations.

A. Comparative Analysis by Scenario

1) *Scenario 1 (Balanced Random)*: Under heterogeneous workloads, FCFS and AJIE present practically equivalent mean turnaround (82318.78 and 82113.28 ticks, respectively — a difference of only 0.25%), both considerably lower than Round Robin's when $C_{sw} > 0$: RR reaches 124061.00 ticks (about 51% higher than AJIE), reflected in 29093.5 context switches — approximately $9.7\times$ the value observed for AJIE (3004.3). The AJIE algorithm reaches an ideal equilibrium point: it preserves the low context-switch frequency characteristic of the non-preemptive regime (25% lower than FCFS/Priority, which each require 4004.3 switches) and achieves a Jain index (90.55%) practically tied with FCFS (90.60%, within the 95% CI margin) and close to Round Robin (95.73%), both far above the fairness collapse of Static Priority (54.45%). That is, in the non-preemptive regime, AJIE delivers the same raw throughput as FCFS without sacrificing fairness equivalent to RR's.

2) *Scenario 2 (I/O-Bound)*: Under intense peripheral contention and short computational bursts, the ready queue exhibits accelerated turnover. Since CPU bursts rarely exceed the quantum ($q = 10$), Round Robin behaves similarly to non-preemptive policies, generating fewer time-based preemptions: its context-switch count (17264.5) approaches that of AJIE (10503.0), with a ratio of only $1.64\times$ — the smallest among the four evaluated scenarios. In this scenario, FCFS's turnaround (60461.61 ticks) is considerably close to AJIE's (63253.79 ticks) — a difference of only 4.4% — so the raw-throughput advantage that characterizes FCFS in I/O-dominated workloads becomes much narrower than in the other scenarios. Even so, AJIE directly benefits interactive processes returning from I/O, avoiding backlog, and even surpasses all other algorithms in fairness, with a Jain index of 98.02% (versus 97.68% for FCFS, 97.27% for RR, and only 67.28% for Static Priority).

3) *Scenario 3 (CPU-Bound)*: With bursts as long as 200 ticks, the convoy effect manifests prominently in FCFS. In Round Robin, the constant slicing of long computational tasks causes an exponential explosion in the context-switch count ($N_{sw} = 60747.6$, i.e., $60.8\times$ more switches than AJIE, which registers only 998.9), accumulating substantial C_{sw} costs in total turnaround: RR reaches 252854.23 ticks, 78.9% above AJIE (141326.83 ticks), and a mean slowdown of 1112.478 —

44.7% higher than AJIE’s slowdown (615.082). By remaining non-preemptive, AJIE does not inflate context switches and maintains maximum processor throughput, even surpassing Static Priority in mean slowdown (615.082 versus 663.348), despite the latter presenting the lowest absolute turnaround (116147.66 ticks) at the cost of a severe loss of fairness (Jain of only 46.97%).

4) *Scenario 4 (Unbalanced Priorities)*: This scenario exposes the critical weakness of Static Priority: with 85% of processes at high priority, the 15% of low-priority processes suffer prolonged retention, collapsing the Jain index to 54.65% — the lowest level among all algorithms and scenarios, except for the CPU-Bound scenario itself. The AJIE algorithm completely neutralizes this collapse: as low-priority processes wait, their systematic aging (+1 at each dispatch) raises their effective priority up to $L = 10$, ensuring guaranteed service and raising the Jain index to 90.72% — an absolute gain of **36.07 percentage points** and a relative fairness gain of **66.0%** over Static Priority, achieved with only 2999.9 context switches (nearly the same amount as the non-preemptive reference regime, and $9.68\times$ fewer than the 29050.4 switches required by Round Robin to obtain only a marginally higher Jain index, 95.74%).

B. Critical Evaluation and Limitations of the AJIE Algorithm

Addressing the assignment’s evaluation criteria:

- **Best-Performing Scenarios:** AJIE stands out markedly in Scenario 4 (Unbalanced Priorities), with a relative fairness gain of 66.0% over Static Priority, and in Scenario 2 (I/O-Bound), where it surpasses even Round Robin in Jain index (98.02% versus 97.27%), showing that the coexistence of priorities requires starvation mitigation at low hardware cost.
- **Optimized Metrics:** Maximization of fairness (J_{slowdown}) combined with strict containment of the number of context switches: aggregating the means across the four workloads (Table III), Round Robin performs about $7.8\times$ more context switches than AJIE overall, reaching a peak of $60.8\times$ in the CPU-Bound scenario.
- **Costs and Limitations Introduced:** Under continuous saturation by high-priority processes, the promotion of aged tasks can marginally raise the waiting time of newly arrived high-priority processes. This is a conscious cost for preserving deterministic corporate SLAs. This cost remains evident relative to Static Priority, which continues to present the lowest absolute turnaround among the four algorithms in every scenario (for example, 56770.92 ticks versus 82113.28 ticks for AJIE in Scenario 1) at the cost of a severe loss of fairness; relative to FCFS, however, the cost ceases to manifest in any relevant way, with AJIE’s turnaround practically equivalent to FCFS’s in scenarios without induced starvation (for example, 82113.28 versus 82318.78 ticks in Scenario 1, a difference of only 0.25%).

C. Impact of I/O Modeling and Context Switching

Modeling multiple I/O devices in parallel via a *Min-Heap* ensured that blocked processes did not generate cross-hardware contention, strictly isolating CPU scheduler efficiency. Adopting $C_{\text{sw}} > 0$ proved to be a determining factor: purely theoretical algorithms that assume zero cost overestimate Round Robin’s performance. Since the slowdown calculation uses T_{ideal} (free of C_{sw}), preemptive policies were duly penalized for the CPU cycles wasted on context switching.

VII. CONCLUSION AND FUTURE WORK

This paper presented the design and empirical analysis of a discrete-event scheduling simulator written in C, comparatively validating the classical FCFS, Round Robin, and Static Priority policies against the proposed AJIE algorithm. Through a rigorous methodology comprising 1,600 seeded simulations and 95% confidence intervals, it was demonstrated that AJIE successfully resolves starvation in non-preemptive environments without resorting to future-execution-time oracles.

The quantitative results consolidate this conclusion: in the unbalanced-priority scenario, AJIE raises the Jain index from 54.65% to 90.72% relative to Static Priority — a relative fairness gain of 66.0% — while in the I/O-Bound scenario it even surpasses Round Robin itself in fairness (98.02% versus 97.27%). This level of fairness is achieved with substantially lower context-switch overhead: aggregating the four workloads, Round Robin performs about $7.8\times$ more context switches than AJIE, reaching $60.8\times$ in the CPU-Bound scenario, where AJIE also delivers 78.9% lower turnaround and 44.7% lower slowdown than RR. Additionally, AJIE now matches FCFS in raw throughput in scenarios without induced starvation (turnaround difference below 0.3% in the Balanced Random scenario), reinforcing that its fairness overhead relative to simple non-preemptive policies is, in practice, negligible. The proposed algorithm proved especially advantageous for batch-processing systems, HPC infrastructures, and atomic pipelines, providing auditable mathematical bounds on task deferral with minimal processing overhead.

REFERENCES

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: John Wiley & Sons, 2018.
- [2] A. S. Tanenbaum and H. Bos, *Modern Operating Systems*, 4th ed. Boston, MA: Pearson, 2015.
- [3] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Pearson, 2018.
- [4] L. Kleinrock, *Queueing Systems, Volume II: Computer Applications*. John Wiley & Sons, 1975.
- [5] F. J. Corbató, M. Merwin-Daggett, and R. C. Daley, “An experimental time-sharing system,” *AFIPS Conference Proceedings (Spring Joint Computer Conference)*, vol. 21, pp. 335–344, 1962.
- [6] A. B. Yoo, M. A. Jette, and M. Grondona, “SLURM: Simple linux utility for resource management,” in *Job Scheduling Strategies for Parallel Processing (JSSPP)*, ser. Lecture Notes in Computer Science, vol. 2862. Springer, 2003, pp. 44–60.
- [7] R. Jain, D.-M. Chiu, and W. R. Hawe, “A quantitative measure of fairness and permutation invariance for resource allocation in shared computer systems,” Digital Equipment Corporation, Tech. Rep. DEC-TR-301, 1984.
- [8] D. Blackman and S. Vigna, “Scrambled linear pseudorandom number generators,” *ACM Transactions on Mathematical Software (TOMS)*, vol. 47, no. 4, pp. 1–32, 2021.